# Estructuras de Datos

2026
Primer Cuatrimestre

# REPASO

# TDA Grafo no Dirigido

```
public interface Graph<V, E> {
        public Iterable<Vertex<V>> vertices();
        public Iterable<Edge<E>> edges();
        public Iterable<Edge<E>> incidentEdges(Vertex<V> v);
        public Vertex<V> opposite(Vertex<V> v, Edge<E> e);
        public Vertex<V> [] endVertices(Edge<E> e);
        public boolean areAdjacent(Vertex<V> v, Vertex<V> w);
        public V replace( Vertex<V> v, V x);
        public E replace( Edge<E> e, E x);
        public Vertex<V> insertVertex(V x);
        public Edge<E> insertEdge(Vertex<V> v, Vertex<V> w, E x);
        public V removeVertex(Vertex<V> v);
        public E removeEdge(Edge<E> e); }
```

# Aclaración

- Debo chequear el vértice (Vertex) o arco (Edge), antes de castearlo a la clase implementada (similar al checkposition)
  - checkVertex(Vertex<V> v)){

```
Vertice<V,E> ver;
if(listaVertices.size()==0 || v==null)
        throw new InvalidVertexException("Vértice inválido");
try {
        ver=(Vertice<V,E>)v;
}
catch(ClassCastException e) { throw new InvalidVertexException("Vértice inválido");}
return ver;
```

  }
  - checkEdge( Edge<E> e)

# En Profundidad (DFS)

```
Algoritmo DFSShell( G : Grafo )
    para cada vértice v de G hacer
        marcar v como no visitado
    para cada vértice v de G hacer
        si v no está visitado entonces
            DFS( G, v )
```

$T(n,m) \rightarrow O(n+m)$

# En Profundidad (DFS)

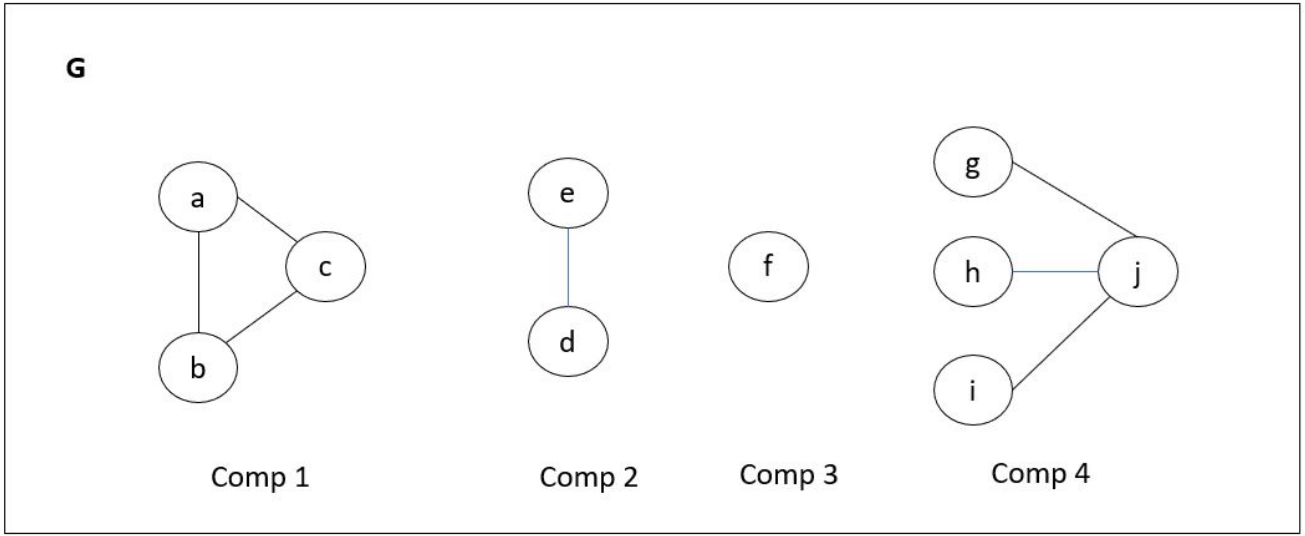
```
Algoritmo DFS( G : Grafo; v : Vértice )
    procesamiento de v previo al recorrido (visita pre)
    marcar a v como visitado
    para cada vértice w adyacente a v en G hacer
        si w no está visitado entonces DFS( G, w )
    procesamiento de v posterior al recorrido  (visita pos)
```

# PROBLEMAS

# Problema 1

Implemente en Java un método tal que dado un grafo no dirigido genérico G retorne un mapeo con sus componentes conexas numeradas y los vértices que pertenecen a esa componente.

# Ejemplo




M={(1,[a,b,c]),(2,[e,d]),(3,[f]),(4,[g,h,i,j])}

# Estrategia

```
Algoritmo DFSShell( G : Grafo )
    para cada vértice v de G hacer
        marcar v como no visitado
    para cada vértice v de G hacer
        si v no está visitado entonces
            DFS( G, v )
```

} Visito todos los vértices de la componente conexa de G que contiene a v

# Estrategia

- Uso como base el recorrido por profundidad (DFS)
- En el DFS Shell
  - Antes de la primera iteración inicializo el mapeo y el contador
  - Antes de visitar una componente conexa inicializo una lista vacía y se la paso a DFS para que me agregue todos los vértices visitados
  - Al salir de la visita de la componente conexa agrego la lista al mapeo con el contador de componentes conexas
- En el DFS
  - Al visitar cada nodo, lo agrego a la lista pasada como argumento

```
Algoritmo HallarComponentesConexas ( G : Grafo ) : Mapeo<Integer, Iterable<Vertex>>
    para cada vértice v de G hacer
        marcar v como no visitado
    Mapeo <...> resultado = new Mapeo<...>;
    int nroComponenteConexa = 0;
```

```
Algoritmo HallarComponentesConexas ( G : Grafo ) : Mapeo<Integer, Iterable<Vertex>>
    para cada vértice v de G hacer
        marcar v como no visitado
    Mapeo <...> resultado = new Mapeo<...>;
    int nroComponenteConexa = 0;
    PositionList<Vertex> verticesComponenteConexa;
    para cada vértice v de G hacer
        si v no está visitado entonces
            nroComponenteConexa++;
            verticesComponenteConexa = new ListaDoblementeEnlazada<Vertex>();
            HallarVerticesComponenteConexaActualConDFS( G, v, verticesComponenteConexa);
            resultado.put(nroComponenteConexa, verticesComponenteConexa);
    return resultado;
```

```
Algoritmo HallarVerticesComponenteConexaActualConDFS( G : Grafo; v : Vértice;
PositionList<Vertex> lista )
    lista.addLast( v );
    marcar a v como visitado;
    para cada vértice w adyacente a v en G hacer
        si w no está visitado entonces
            HallarVerticesComponenteConexaActualConDFS( G, w, lista )
```

¿Qué pasa si lo agrego al final del algoritmo?

# Problema 2

Implemente en Java un método tal que dado un grafo no dirigido genérico G  y dos vértices, retorne un camino entre ambos vértices.
Si no existe un camino entre los vértices dados el método debe retornar nulo.

# Estrategia

- Origen y destino tienen que estar en la misma componente conexa
  - El DFS shell no necesita un bucle para empezar desde otro vértice
- Necesito una lista para pasarle al DFS y que me añada los vértices del camino
- El DFS debe devolver un booleano que indique si se encontró o no el camino
- Dentro del DFS (grafo, origen, destino y lista)
  - Marco origen como visitado
  - Añado origen a la lista
  - Caso base (origen es igual a destino)
    - aviso que encontré
  - Caso recursivo (origen no es igual a destino)
    - Para cada adyacente $w_i$ no visitado siempre y cuando no haya encontrado camino
      - Llamo a DFS (grafo, $w_i$, destino, lista)
    - Me fijo qué pasó con los adyacentes
      - Si encontró camino, devuelvo true
      - Si no encontró camino,
        - ¿saco a origen de la lista del camino?
        - ¿saco a $w_i$ de los visitados?

```
Algoritmo CaminoShell( G : Grafo; origen, destino: Vertex) : PositionList<Vertex>
    para cada vértice v de G hacer
        marcar v como no visitado
    PositionList<Vertex> resultado = new ListaDoblementeEnlazada<Vertex>()
    encontreCamino = CaminoDFS( G, origen, destino, resultado)
    si encontreCamino entonces
        return resultado
    else
        return null
```

```
Algoritmo CaminoDFS( G : Grafo; origen, destino : Vértice; PositionList<Vertex<V>> lista ) : boolean

    marcar a origen como visitado;
    lista.addLast( origen );
    if (origen == destino ) {
        return true;
    }
    else {
        encontre = false;
        para cada vértice w adyacente a origen en G AND !encontre hacer
            si w no está visitado entonces
                encontre = CaminoDFS (G, w, destino, lista)

        si encontre {
            return true;
        }
        sino {
            lista.remove( lista.last() );
            return false;
        }
    }
```

# Break.

# Problema 3

Implemente en Java un método tal que dado un grafo no dirigido genérico G  y dos vértices, retorne un listado de todos los caminos entre ambos vértices.
Si no existe un camino entre los vértices dados el método debe retornar una lista nula.

# Estrategia

¿En qué cambia con el algoritmo anterior?

- El DFSShell debe devolver una lista de listas de caminos (todos)
- El DFS
  - debe manejar el camino actual (camino) y la lista de todos los caminos (todos)
  - Caso base
    - Si origen == destino
      - Se debe copiar/clonar el camino en la lista de todos
    - Después de buscar en los adyacentes
      - ¿Borro el último camino de todos dependiendo del éxito de la búsqueda?
      - ¿Saco al nodo origen del camino?
      - ¿Saco al nodo origen de los visitados?

```
Algoritmo CaminosShell( G : Graph<V,E>; origen, destino: Vertex<V>) :
                    PositionList<PositionList<Vertex<V>>>

    para cada vértice v de G hacer

        marcar v como no visitado

    PositionList<Vertex> camino = new Lista<Vertex<V>>();

    PositionList<PositionList<Vertex<V>>> todos =

        new Lista<PositionList<Vertex<V>>>();

    CaminosDFS( G, origen, destino, todos, camino );

    return todos;
```

Algoritmo CaminosDFS( G : Graph<V,E>; origen, destino : Vértice; PositionList<PositionList<Vertex<V>>> todos; PositionList<Vertex<V>> camino )

```
marcar a origen como visitado;

camino.addLast( origen );

if (origen == destino )
        todos.addLast(clonar(camino))
```

```
Algoritmo CaminosDFS( G : Graph<V,E>; origen, destino : Vértice; PositionList<PositionList<Vertex<V>>> todos;
PositionList<Vertex<V>> camino )
    marcar a origen como visitado;
    camino.addLast( origen );
    if (origen == destino )
        todos.addLast(clonar(camino))
    else
        para cada vértice w adyacente a origen en G hacer
            si w no está visitado entonces
                CaminosDFS( G, w, destino, todos, camino)
```

Algoritmo CaminosDFS( G : Graph<V,E>; origen, destino : Vértice; PositionList<PositionList<Vertex<V>>> todos; PositionList<Vertex<V>> camino )

```
marcar a origen como visitado;

camino.addLast( origen );

if (origen == destino )
        todos.addLast(clonar(camino))
else

        para cada vértice w adyacente a origen en G hacer

                si w no está visitado entonces

                        CaminosDFS( G, w, destino, todos, camino)

camino.remove( camino.last() )

marcar a origen como NO visitado
```

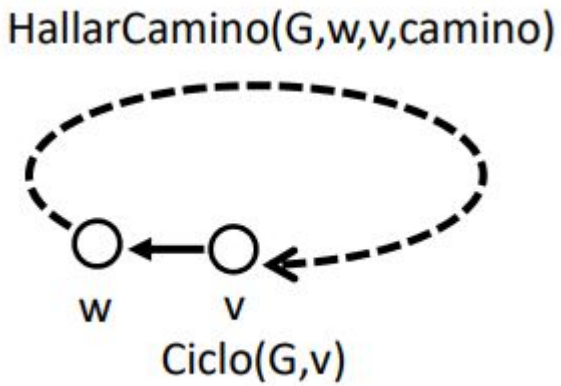
# Problema 4

Determinar si en un grafo dirigido G existe un ciclo que contenga a un vértice v.

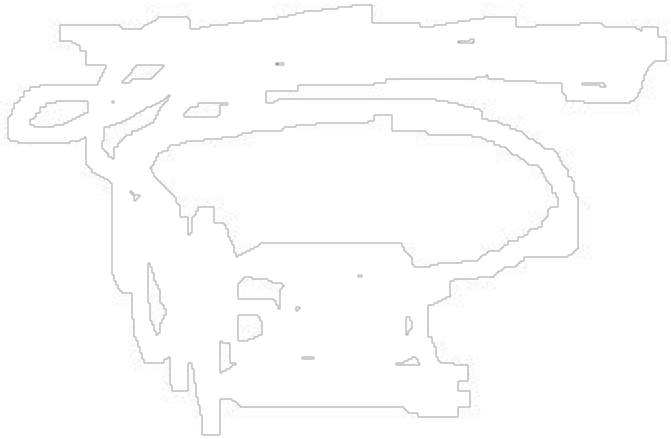
# Problema 4

Determinar si en un grafo dirigido G existe un ciclo que contenga a un vértice v.

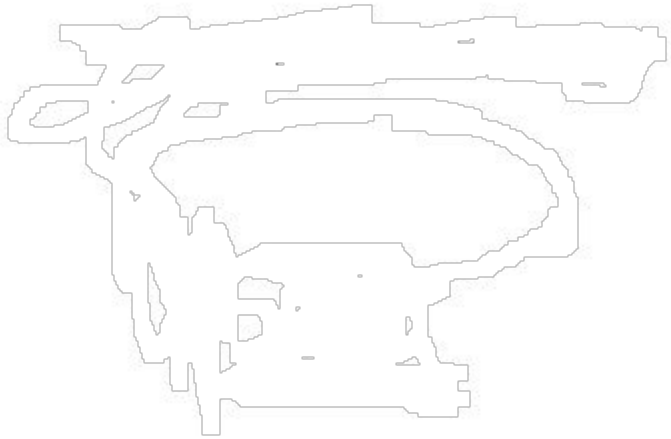
La estrategia consiste en buscar caminos desde los adyacentes de v (que se llaman w) hacia v.



```
Algoritmo HallarCiclo( G, v ) : Lista
  encontre ← falso
  mientras hay adyacentes para considerar y no encontre hacer
        w ← siguiente adyacente de v en G
        encontre ←  CaminoDFS  ( G, w, v, camino)
 finmientras
 ciclo ← new Lista()
 si encontre  entonces
     ciclo.addLast( v )
     para cada vertice x de camino hacer
        ciclo.addLast( x )
     finpara
  finsi
  retornar ciclo
```



```
Algoritmo HallarCiclo( G, v ) : Lista
  encontre ← falso
  mientras hay adyacentes para considerar y no encontre hacer
        w ← siguiente adyacente de v en G
        encontre ←  CaminoDFS  ( G, w, v, camino)
 finmientras
 ciclo ← new Lista()
 si encontre  entonces
     ciclo.addLast( v )
     para cada vertice x de camino hacer
        ciclo.addLast( x )
     finpara
 finsi
 retornar ciclo
```



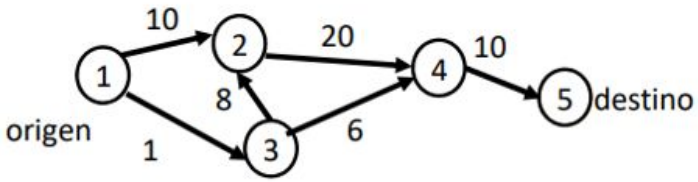
$T_{hallarciclo}(n,m) = O(grado(v)*(n+m) + n)$

# Problema 5

Dado un grafo G con vértices rotulados con caracteres y arcos rotulados con números enteros y dos vértices origen y destino, escriba un método que permita hallar el camino de costo mínimo entre los vértices dados.

# Estrategia

- Aplicar la estrategia del algoritmo buscar todos los caminos
- Diferencias
  - En lugar de mantener todos los caminos, mantener el actual y el mejor (mínimo)
  - Tengo que llevar la cuenta del costo (el costo está en las aristas)
  - Cada vez que encuentro un nuevo camino lo comparo con el mejor
  - Si el camino actual es mejor, actualizo el mejor camino

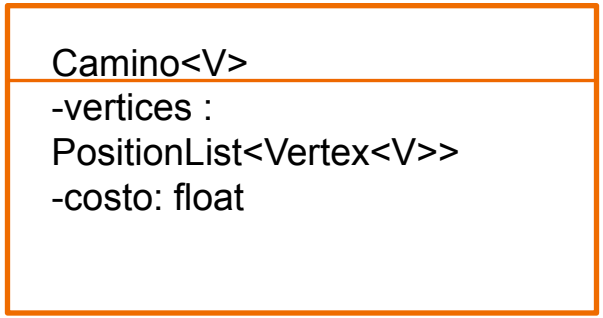



*El camino 1-2-4-5 tiene costo 10+20+10=40*
*El camino 1-3-2-4-5 tiene costo 1+8+20+10=39*
*El camino 1-3-4-5 tienen costo 1+6+10=17*
*Para explorar el segundo y tercer caminos es necesario desmarcar los vértices ya visitados al generar el primer camino.*

```
public class Camino<V> {
    protected PositionList<Vertex<V>> vertices;
    protected float costo;
    public Camino() {
            this.camino = new ListaDobleEnlazada<Vertex<V>>();
            this.costo = 0.0f; }
    public PositionList<Vertex> getVertices()
    public void setVertices(PositionList<Vertex<V>> nuevo)
    public void setCosto (int costo) { this.costo=costo; }
    public int getCosto( ) {return this.costo;}
```

Algoritmo CaminoMínimoShell( G : Grafo; origen, destino: Vertex) : PositionList<Vertex>

```
INFINITO = Float.MAX_VALUE
para cada vértice v de G hacer
        marcar v como no visitado
Camino resultado = new Camino()
Camino actual = new Camino()
resultado.setCosto(INFINITO)
CaminoMinimoDFS( G, origen, destino, resultado, actual);
return resultado.getVertices();
```



Algoritmo CaminoMinimoDFS( G : Grafo; origen, destino : Vértice; resultado, actual : Camino)

```
marcar a origen como visitado;
actual.getCamino().addLast( origen );

if (origen == destino )

        if( actual.getCosto() < resultado.getCosto() )
                resultado.setCamino( clonar ( actual.getCamino() ) );
                resultado.setCosto( actual.getCosto() );

else

        para cada vértice w adyacente a origen en G a través del arco e hacer

                si w no está visitado entonces

                        actual.setCosto( actual.getCosto() + e.element() );
                        CaminoMinimoDFS( G, w, destino, resultado, actual)
                        actual.setCosto (actual.getCosto() - e.element() );

        camino.remove( camino.last() )
        marcar a origen como NO visitado
```

# Para pensar...

¿Qué cambiaría en estos ejercicios en el caso de que el grafo G sea dirigido?

# THE END.